ФУНКЦІОНАЛЬНІ ВИМОГИ ДЛЯ TICKETPULSE

Нижче представлений **повний** та **глибоко деталізований** набір функціональних вимог до системи управління NFT-квитками TicketPulse.

---

5 ФУНКЦІЇ ПРОДУКТУ

5.0 Загальний функціонал

Перед взаємодією з основним функціоналом користувач потрапляє на головну сторінку, має можливість ознайомитися з принципами роботи Web3 та обирає сценарій використання (роль).

F-0.1: Головна сторінка та загальна інформація (Landing Page)

Опис: Точка входу для всіх неавторизованих користувачів (роль "Гість"). Сторінка має презентувати продукт, пояснювати його цінність та надавати відповіді на поширені запитання.

Передумови: Користувач відкрив веб-додаток за основним URL.

Структура та зміст:

1. **Hero-секція:** Короткий, зрозумілий опис платформи (value proposition) та заклики до дії: "Стати організатором" та "Купити квиток".
2. **Опис можливостей:** Блок з описом ключових переваг (прозорість, неможливість підробки квитків, вторинний ринок тощо).
3. **FAQ (Питання та відповіді):** Розділ, що містить відповіді на базові питання (наприклад, "Що таке NFT-квиток?", "Чи можна повернути квиток?", "Які комісії?").
4. **Футер:** Посилання на соціальні мережі, технічну підтримку та юридичну інформацію.

Результат: Користувач отримав базове розуміння продукту і бачить точки входу для подальшої взаємодії.

F-0.2: База знань Web3 (Інструкція з використання гаманців)

Опис: Освітній розділ для користувачів, які не мають досвіду роботи з блокчейн-технологіями. Мета — знизити поріг входу.

Доступ: Через посилання в меню навігації ("Допомога" або "Як це працює").

Функціональні вимоги до контенту:

1. **Типи гаманців:** Пояснення простою мовою, які гаманці підтримуються (наприклад, MetaMask, Phantom, WalletConnect) з прямими посиланнями на офіційні джерела завантаження.
2. **Процес встановлення:** Покроковий гайд зі скріншотами (або слайдер/карусель), що демонструє процес встановлення розширення браузера або мобільного додатку.
3. **Основи використання:**
    - **Баланс:** Інструкція, де дивитися баланс нативних токенів для оплати квитку та комісій (Gas).
    - **NFT:** Пояснення, у якій вкладці гаманця шукати придбані квитки (розділ NFTs/Collectibles).
    - **Підписання транзакцій:** Пояснення, що таке "підпис" (Signature) та чому це безпечно (розрізнення між наданням доступу до коштів та простою авторизацією).

Результат: Користувач розуміє, який інструмент йому потрібен, встановив його та готовий до підключення.

F-0.3: Вибір ролі користувача (сценарію взаємодії)

Опис: Перший активний крок взаємодії. Користувач визначає свій намір (сценарій використання) натисканням відповідних елементів інтерфейсу в різних зонах екрану. Це ініціює єдиний процес підключення.

Передумови: Користувач знаходиться на Головній сторінці з загальною інформацією (F-0.1).

Процес виконання:

1. Система відображає елементи вибору ролі у двох зонах:
    - **У хедері:** постійно доступні кнопки навігації "Вхід для організаторів" та "Вхід для покупців".
    - **У hero-секції:** великі акцентні кнопки "Створити подію" (відповідає ролі організатора) та "Знайти квитки" (відповідає ролі покупця).
2. Користувач натискає будь-яку з цих кнопок.
3. Система фіксує обраний намір у тимчасовому стані додатка.
4. Система автоматично переходить до процесу F-0.4 (Підключення криптогаманця).

Результат: Система чітко розрізняє намір користувача (купівля чи створення) незалежно від того, де він натиснув кнопку (у меню чи на банері), і готова завантажити відповідний інтерфейс.

F-0.4: Підключення криптогаманця (вхід в систему)

Опис: Технічний процес ідентифікації користувача через Web3-гаманець. Цей процес запускається автоматично після виконання кроку F-0.3.

Підтримка різних типів гаманців:

- **EOA-гаманці:** Стандартні криптографічні гаманці, керовані приватним ключем.

Процес виконання:

1. **Відображення опцій підключення:** Система відкриває модальне вікно "Підключити гаманець", яке пропонує повний перелік варіантів (незалежно від встановлених розширень):
    - **Browser Extension:** MetaMask, Phantom.
        - *Логіка відображення:* Якщо розширення виявлено у браузері, воно підсвічується як "Встановлене", але інші опції не приховуються, щоб зберегти можливість вибору.
    - **WalletConnect:** Універсальний протокол для підключення будь-якого мобільного гаманця.
    - **Довідка:** У нижній частині модального вікна *завжди* доступне посилання: "Немає гаманця або не знаєте як користуватися? Читати інструкцію" (перехід до F-0.2).
    - **Примітка для апаратних гаманців:** Якщо ви використовуєте апаратний гаманець (Ledger, Trezor), будь ласка, спочатку підключіть його до обраного вами програмного гаманця (наприклад, MetaMask, Trust Wallet, Ledger Live), а потім оберіть цей програмний гаманець зі списку вище.
2. **Логіка вибору методу:**
    - **Сценарій А (browser extension):** Якщо користувач обирає цей метод, система ініціює запит на отримання публічної адреси.
    - **Сценарій Б (WalletConnect):** Система визначає тип пристрою користувача:
        - *Desktop:* Генерується універсальний QR-код для сканування. Користувач може вибрати гаманець зі списку, щоб отримати **брендований QR-код** або **скопіювати Deep Link**..

        - *Mobile: Користувач бачить список гаманців і **натискає на встановлений ним гаманець**. Після цього кліку система використовує **Deep Linking** (URI-схему обраного гаманця) для **автоматичного редіректу** на встановлений додаток гаманця, щоб уникнути необхідності сканувати QR-код самим собою.*

3. **Дії користувача:** У інтерфейсі гаманця користувач обирає акаунт та підтверджує з'єднання. Для ідентифікації жорстке перемикання мережі (Chain Switch) в інтерфейсі часто не є обов'язковим: у гаманці користувача буде повідомлено про назву мережі, якої публічну адресу передається.
4. **Ідентифікація та Перевірка:**
    - Система отримує публічну адресу (наприклад, 0x742d...).
5. **Маршрутизація:** На основі вибору, зробленого в пункті F-0.3, система перенаправляє користувача:
    - **Якщо роль "Організатор":** Перенаправлення в **кабінет організатора зі списком його подій**
    - **Якщо роль "Покупець":** Перенаправлення на **Загальний каталог подій (Marketplace)**.
        - *Примітка:* Доступ до розділу "Мої квитки" для покупця здійснюється окремо через навігаційне меню (хедер) після ідентифікації.
6. **Підписка на зміни:** Система підписується на події гаманця:
    - Зміна активного акаунта (`accountsChanged`)
    - Зміна мережі (`chainChanged`)
    - Відключення гаманця (`disconnect`)

Принцип прив'язки особистості (Digital Artifact): Користувач повинен розуміти, що вхід до події буде здійснюватися за допомогою криптографічного підпису. Цей підпис (або його регенерація в майбутньому) можливий **виключно тим самим гаманцем**, яким було підписано повідомлення при покупці квитка.

- Користувач має ЗБЕРЕГТИ файл підпису ("цифровий артефакт") у пам'яті телефону.
- У разі втрати файлу, його можна згенерувати повторно, але тільки за наявності доступу до **гаманця** покупця, яким було підписано повідомлення при покупці квитка.

Обробка помилок:

- **Відмова:** Якщо користувач закрив вікно — повернення на Головну сторінку.

F-0.5: Відключення гаманця (Вихід із системи)

Опис: Функція для завершення поточної сесії.

Процес виконання:

1. Користувач натискає кнопку “Вийти” в інтерфейсі
2. Система:
   - Очищує збережену адресу з локального стану
   - Видаляє всі кешовані дані користувача
   - Скасовує підписки на події гаманця
3. Користувач повертається до головної сторінки в ролі “Гість”
4. Всі персоналізовані дані (колекція квитків, створені події) стають недоступними

Примітка: Фактичне “відключення” на рівні блокчейну не існує – це лише операція на стороні клієнта. Користувач залишається власником своїх активів, і при повторному підключенні того ж гаманця всі дані знову стануть доступними.

F-0.6: Підготовка та валідація середовища перед підписанням транзакції

Опис: Комплексний механізм валідації, що гарантує виконання транзакцій (створення події, покупка квитка, скасування) виключно у **цільовій мережі** та з **авторизованого акаунту**. Цей крок є обов'язковим пререквізитом для будь-якої дії, що вимагає запису в блокчейн, і запобігає помилкам, пов'язаним з асинхронним перемиканням гаманця.

Передумови: Користувач авторизований (F-0.4) та ініціював дію (натиснув кнопку "Створити", "Купити", "Скасувати").

Процес виконання (послідовний):

Етап 1: Валідація акаунту (Account Consistency Check)

1. **Перевірка поточного підписанта:** Система звертається до провайдера гаманця для отримання адреси, яка на даний момент є активною в гаманці.

2. **Порівняння:** Система порівнює активну адресу гаманця з адресою, збереженою в сесії додатку (User ID з F-0.4).

3. **Логіка розгалуження:**

   - **Збігається:** Перехід до Етапу 2.

- **Не збігається (Detected Account Mismatch):** Система блокує виконання дії та показує модальне вікно попередження: *"Виявлено зміну акаунту. Ви увійшли як [Адреса А], але ваш гаманець перемкнено на [Адреса Б]"*.
    - **Опція 1:** "Повернути попередній акаунт" (Користувач має перемкнути гаманець назад).
    - **Опція 2:** "Увійти новим акаунтом" (Це призведе до перезавантаження сторінки та нової авторизації F-0.4, поточна незавершена дія буде скасована).

Етап 2: Валідація мережі (Network Consistency Check)

1. **Перевірка Chain ID:** Якщо акаунт правильний, система зчитує chainId поточного провайдера.
2. **Порівняння:** Порівнює з ID цільової мережі протоколу.
3. **Логіка розгалуження:**
    - **Варіант А (Мережа правильна):** Система одразу переходить до формування параметрів транзакції.
    - **Варіант Б (Мережа неправильна):** Система ініціює запит на перемикання.

Етап 3: Автоматичне перемикання (Switching Flow)

1. Система викликає метод перемикання мережі з параметрами цільової мережі.
2. Гаманець відображає системне вікно підтвердження.
3. *Якщо мережі немає в гаманці:* Система автоматично викликає додавання мережі.

Результат: Після успішного проходження обох етапів система автоматично продовжує виконання початкової дії (формування транзакції) без необхідності повторного натискання кнопки.

Обробка помилок:

- **Відхилення (User Rejected):** Якщо користувач відмовляється перемикати мережу або акаунт, дія скасовується, і користувач залишається на поточному екрані з повідомленням про помилку.

F-0.7: Обробка життєвого циклу транзакції та зворотній зв'язок

Опис: Механізм взаємодії додатку з транзакцією після того, як користувач підтвердив її у своєму гаманці. Вимога розмежовує відповідальність гаманця (виконання) та додатку (інформування).

Передумови: Користувач успішно пройшов валідацію F-0.6 та натиснув «Підтвердити» у інтерфейсі свого гаманця.

Розподіл відповідальності:

- **Гаманець (Wallet):** Автоматично формує пакет даних, відправляє в RPC, додає запис у власну вкладку "Activity" (Активність), відображає технічні статуси (Pending/Confirmed/Failed). Додаток **не повинен** намагатися дублювати цей інтерфейс всередині гаманця.
- **Додаток (Web App):** Відповідає за контекстне інформування користувача на сторінці сайту, оскільки користувач може закрити вікно гаманця одразу після підписання.

Процес виконання:

1. **Перехоплення Хешу (Hash Capture):**
    - Одразу після підписання транзакції гаманець повертає transactionHash (унікальний ідентифікатор транзакції).
    - **Критична вимога:** Додаток повинен миттєво "зловити" цей хеш, не чекаючи завершення транзакції.
2. **Відображення статусу "В обробці" (Optimistic UI):**
    - Система відображає користувачу повідомлення: *"Транзакцію відправлено в мережу!"*.
    - Повідомлення **обов'язково** містить активне посилання на Block Explorer, сформоване на основі хешу: https://[explorer-url]/tx/[txHash].
    - Інтерфейс блокує кнопку дії (наприклад, "Купити") та показує індикатор завантаження, щоб уникнути подвійного списання.
3. **Моніторинг виконання (Waiting for Receipt):**
    - Додаток у фоновому режимі опитує провайдера, очікуючи на підтвердження включення транзакції у блок.
4. **Фіналізація (Completion):**
    - **Успіх:** Отримавши підтвердження, система змінює статус на *"Успішно"*.
        - *Архітектурна примітка:* Система **не парсить "сирі" логи** (Logs/Events) з тіла транзакції на клієнті. Замість цього, після підтвердження транзакції, фронтенд робить запит до індексатора **The Graph**, щоб отримати вже структуровані оновлені дані (наприклад, метадані нового квитка) та відобразити їх користувачу.

- **Помилка (Revert):** Якщо транзакція відхилена блокчейном:
    - **Декодування (RPC):** Система намагається декодувати причину помилки з ABI смарт-контракту (Custom Errors), щоб показати користувачу зрозумілий текст (наприклад, «Квитки розпродано»).
    - **Перевірка стану (The Graph):** Якщо причина помилки неочевидна, система робить запит до The Graph, щоб перевірити актуальний статус активу (наприклад, чи не був квиток проданий іншому користувачу в цей момент - race condition), і пояснити причину відхилення.
    - Після аналізу система розблоковує інтерфейс для повторної спроби.

Результат: Користувач отримує миттєвий зворотній зв'язок на сайті і має можливість відстежувати прогрес у незалежному джерелі (Explorer), навіть якщо закриє вкладку гаманця.

F-0.8: Глобальний стандарт відображення вартості

Опис: У всіх інтерфейсах системи, де фігурує вартість у нативній криптовалюті (ціна квитка, роялті, сума замовлення), система повинна автоматично відображати еквівалент у фіатній валюті (USD/EUR).

Мета: Забезпечити користувачу зрозумілу оцінку вартості активів, нівелюючи складність сприйняття дробових чисел криптовалюти (наприклад, 0.05 ETH).

- **Принцип "Подвійного відображення" (UI Pattern):**
    - **Основна валюта (Жирним/Акцентним):** Нативний токен мережі (наприклад, **0.05 ETH**). Це єдина валюта, яка фактично списується смарт-контрактом.
    - **Довідкова валюта (Другорядним/Сірим):** Фіатний еквівалент (наприклад, *≈ $135.50*).
- **Технічна реалізація (Source of Truth):**
    - Дані про курси валют отримуються **виключно** через **децентралізований оракул Chainlink Data Feeds**.
    - Система звертається до смарт-контрактів агрегаторів для пар ETH/USD та ETH/EUR.
    - Використання децентралізованого джерела гарантує, що відображена ціна відповідає реальному ринковому курсу, що використовується в екосистемі Web3, та захищена від маніпуляцій централізованих API.
- **Оновлення даних:**

- Курс оновлюється на клієнті при кожному завантаженні сторінки та раз на хвилину, щоб уникнути розбіжностей під час різкої волатильності ринку.

5.1 Управління подіями (функціонал організатора)

Функції, доступні для ролі «організатор заходів».

F-1: Створення події, визначення квитків та правил ринку

Опис: Це атомарна блокчейн-транзакція, яка ініціюється організатором для запуску нової події. Вона розгортає новий клон-контракт події та встановлює всі його початкові параметри.

Передумови:

- Користувач обрав роль “Організатор” (F-0.3)

- Користувач підключив гаманець (F-0.1)

- На балансі достатньо нативної валюти для оплати газу

- На пристрої користувача є підготовлений файл обкладинки (JPG/PNG)

Вхідні дані (форма створення події):

1) Інформація про подію (всі поля обов’язкові):

- **Назва з вказанням артиста**
  Приклад: “The Neighbourhood - The World Tour 2026”
  Тип: рядок, макс. 200 символів

- **Опис для покупців**
  Багаторядкове текстове поле з детальною інформацією про захід
  Тип: рядок, макс. 2000 символів

- **Місце проведення**
  Приклад: “Палац спорту, Київ, Україна”
  Тип: рядок, макс. 300 символів

- **Дата і час початку**
  Зберігається як Unix timestamp (UTC)
  Валідація: не може бути в минулому

- **Дата і час завершення**
  Зберігається як Unix timestamp (UTC)
  Валідація: має бути пізніше за час початку

- **Обкладинка події**
  IPFS CID
  Рекомендований формат: JPG/PNG
  Рекомендована роздільність: мінімум 1200x630 px
  Максимальний розмір файлу: 5 МБ

2) Параметри квитків та ринку для одного типу квитків (всі поля обов'язкові):

- **Назва типу квитка**
  Приклад: "VIP", "General Admission", "Early Bird"
  Тип: рядок, макс. 50 символів

- **Унікальний символ NFT-колекції**
  Короткий ідентифікатор (ticker), наприклад: "TNWT26"
  Тип: рядок, 2-10 символів, лише великі латинські літери та цифри
  Відображення короткого коду активу в інтерфейсі криптогаманців користувачів, а також для ідентифікації у блокчейн-експлорерах.

- **Загальна кількість (максимальна емісія)**
  Скільки всього квитків буде випущено для цієї події
  Тип: ціле число, мінімум 1, максимум 10,000,000 (Найбільшим концертом в історії вважається виступ **Рода Стюарта** в Ріо-де-Жанейро 31 грудня 1994 року, який зібрав рекордні **4 мільйони людей**)
  Після створення події змінити неможливо

- **Номінальна ціна первинного продажу**
  Ціна в нативній валюті мережі
  Введення ціни в нативній валюті мережі, конвертація у найменші одиниці (wei) відбувається автоматично перед відправкою транзакції
  Тип: десяткове число, мінімум > 0
  Приклад: 0.05 ETH

- **Відсоток роялті з кожного перепродажу**
  Частка від повної ціни вторинного продажу, яка автоматично відраховується організатору. Частка – не від націнки, бо при частці від повної ціни навіть якщо квиток перепродують за номіналом (без націнки), організатор все одно отримує роялті з перепродажу. Також у сфері блокчейн та NFT роялті завжди автоматично вираховується із загальної суми транзакції.
  Тип: ціле число, від 0% до 20%
  Приклад: 10% означає, що з кожної перепродажі 10% йде організатору, 90% – продавцю
  Рекомендація: 5-15%

- **Єдиний фіксований % націнки для вторинного ринку**
  Максимально дозволена націнка на перепродаж відносно номінальної ціни
  Тип: ціле число, від 0% до 100%
  Приклад: 20% означає, що якщо первинна ціна 0.05 ETH, то ціна перепродажу буде 0.06 ETH

Ця ціна жорстко фіксована смарт-контрактом – покупець на вторинному ринку завжди платить `номінальна_ціна * (1 + націнка/100)`

- **Максимальна кількість квитків на один гаманець**
  Обмеження для боротьби з масовим скальпінгом
  Тип: ціле число, мінімум 1, максимум – загальна кількість
  Приклад: 4 квитки
  Валідація виконується смарт-контрактом при кожній спробі купівлі

Процес виконання:

1. **Валідація введених даних на клієнті:**
   - Перевірка обов'язкових полів
   - Перевірка логічності дат (завершення після початку)
   - Перевірка числових діапазонів
2. **Завантаження обкладинки та даних події в IPFS**:
   - Показується індикатор прогресу
   - Після завантаження отримується IPFS CID обкладинки та даних події
   - У разі помилки користувачу пропонується повторити спробу
3. **Підготовка стратегії метаданих (On-Chain Generation):**

   На відміну від класичного підходу з генерацією тисяч JSON-файлів, система **не створює** окремі файли метаданих для кожного квитка.

   Замість цього, формуються параметри для смарт-контракту, який буде генерувати JSON динамічно (Dynamic Base64 Encoding) в момент запиту. Це дозволяє уникнути обмежень IPFS на кількість файлів та економить час/гроші.

4. **Підготовка параметрів для смарт-контракту:**
   - Конвертація ціни з ETH у wei
   - Упакування всіх параметрів у структуру для передачі в метод контракту
5. **Формування транзакції:** Система викликає метод фабричного контракту для розгортання нового **Proxy-контракту (EIP-1167)**, передаючи туди необхідні налаштування та CID даних події(JSON з даними подіями включаючи imageCID).
6. **Підпис транзакції користувачем:**
   - Гаманець відкриває вікно підтвердження
   - Відображається оцінка вартості газу
   - Користувач підтверджує або відхиляє
7. **Очікування підтвердження:**

- Транзакція відправлена в мемпул
- Інтерфейс показує індикатор “Створення події…” з хешем транзакції та посиланням на block explorer
- Після включення транзакції в блок (1-3 підтвердження) подія вважається створеною

8. **Результат:**

- З блокчейну отримується адреса нового контракту події
- Подія миттєво стає активною для первинного продажу
- Система індексує подію для відображення в каталозі (F-5)
- Організатор автоматично перенаправляється на сторінку своєї нової події (F-3)

Функціональне розширення: Двосторонній конвертер валют (Chainlink Oracle)
Для зручності організатора та уникнення помилок при оцінці вартості, поле вводу ціни оснащене вбудованим калькулятором-конвертером.

1. **Джерело даних (Source of Truth):**
    - Курси валют (ETH/USD, ETH/EUR) отримуються в реальному часі через децентралізовану мережу оракулів **Chainlink Data Feeds**.
    - Система виконує прямий read-запит до смарт-контракту *Chainlink Aggregator* (метод latestRoundData), оминаючи централізовані API (як CoinGecko), що гарантує відображення "офіційної" ончейн-ціни активу.
2. **Сценарії взаємодії (Двосторонній режим):**
    - **Режим А (Введення крипти):** Організатор вводить 0.05 ETH.
        - Система миттєво відображає еквівалент: *"≈ $135.50 / €128.20 (за курсом Chainlink Oracle)"*.
    - **Режим Б (Введення фіату):** Організатор вводить бажану суму отримання 100 USD.
        - Система автоматично розраховує та підставляє необхідну суму в крипті: 0.0369 ETH.
        - Відображається попередження: *"У смарт-контракт буде записано фіксоване значення 0.0369 ETH. Фактична ціна в USD для покупця може коливатися разом з курсом ринку"*.

Підтримка стандартів метаданих для відображення у гаманцях:

Для забезпечення бездоганного користувацького досвіду, система реалізує три рівні метаданих:

1. Рівень токена (On-Chain tokenURI)

Щоб уникнути проблем із завантаженням мільйонів файлів, смарт-контракт реалізує метод tokenURI(tokenId), який **динамічно генерує** JSON-рядок, кодує його в Base64 і повертає гаманцю як data-URI.

Структура, яку генерує контракт:

```
{
  "name": "The Neighbourhood - The World Tour 2026 – Ticket #1234",
  "description": "Official Ticket",
  "image": "ipfs://[CID обкладинки]",
  "external_url": "https://ticketpulse.app/event/[ID]",
  "attributes": [
    {"trait_type": "Type", "value": "VIP"},
    {"trait_type": "Start Date", "value": "2026-05-15"}
  ]
}
```

- **Динамічна назва:** Контракт автоматично підставляє номер квитка (#1234) у поле name, щоб у гаманці кожен квиток виглядав унікально.
- **Спільна картинка:** Всі квитки посилаються на один imageCID.

2. Рівень колекції (Contract Metadata):

Щоб квитки в гаманці не виглядали як "Unknown Contract", а групувалися під логотипом події, контракт повинен реалізовувати метод contractURI(). Сучасні гаманці використовують ці дані для оформлення заголовка групи токенів:

JSON

```
{
  "name": "The Neighbourhood - The World Tour 2026 Tickets",
  "description": "Official NFT tickets for the concert",
  "image": "ipfs://[CID]",
  "external_link": "https://ticketpulse.app/event/[ID]",
  "seller_fee_basis_points": 1000,  // 10% royalty
  "fee_recipient": "0x[організатор адреса]"
}
```

3. Примусове додавання (EIP-747):

Оскільки гаманці не завжди автоматично "бачать" нові NFT (особливо в нових мережах), інтерфейс повинен реалізовувати метод wallet_watchAsset. Це дозволяє користувачу натисканням однієї кнопки "Додати в гаманець" на сайті змусити додаток гаманця відстежувати цей конкретний квиток.

Життєвий цикл події (автоматичні фази):

Статус події визначається не явним полем у контракті, а **реальним часом** (timestamp блоку) відносно дат початку/завершення:

Фаза 1: Продаж квитків (від створення до `startTime`)

- Дозволена первинна купівля (F-6)

- Дозволений вторинний ринок (F-14–F-22)

- Виведення коштів організатором **заблоковано**

Фаза 2: Тривання події (від `startTime` до `endTime`)

- Усі ринкові операції **повністю заморожені**

- Первинна купівля заблокована

- Виставлення/зняття з продажу заблоковано

- Постановка в чергу на купівлю заблокована

- Квитки, що вже були у черзі на продаж, залишаються функціональними для входу (QR-код генерується)

- Створюється статичний snapshot бази власників для офлайн-валідації (F-26)

Фаза 3: Колекція та лояльність (після `endTime`)

- Первинна купівля та вторинний ринок **остаточно закриті**

- Організатор може вивести всі заробітки (F-4)

- Квиток перетворюється на **колекційний цифровий актив**

- Відкриваються можливості для програм лояльності у майбутньому:

    - Організатор може надавати власникам квитків ексклюзивні бонуси

    - Доступ до закритих івентів для фанатів

    - Знижки на майбутні події

    - Вибіркове винагородження лише тих, хто фізично відвідав захід (синхронізація даних про використання)

Обмеження поточної версії:

- Підтримується лише **один тип квитка** на подію (без поділу на VIP/Standard/Early Bird)

- Після створення параметри події **незмінні** (на рівні смарт-контракту)

- Відміна події можлива лише шляхом розгортання окремої логіки повернення коштів (не реалізовано в базовій версії)

Використання EIP-1167 Minimal Proxy Clones (для оптимізації газу):

Так як буде створюватися велика кількість подій, використовуватиметься патерн клонування контрактів (EIP-1167), де:

- Розгортається один “майстер-контракт” з усією логікою

- Для кожної нової події створюється легкий проксі-контракт, що делегує виклики до майстра

- Вартість створення події знижується у декілька разів

F-2: Перегляд списку створених подій

Опис: Організатор має можливість у власному кабінеті бачити список усіх подій, де адреса його гаманця вказана як творець.

Передумови:

- Користувач підключив гаманець (F-0.1)

- Обрав роль “Організатор” (F-0.3)

Процес виконання:

1. **Запит даних:**
   - Система виконує GraphQL-запит до індексера, фільтруючи події за адресою організатора.
2. **Відображення:**
   - Список виводиться у вигляді **карток**
3. **Порожній стан:**
   - Якщо організатор ще не створив жодної події, відображається:
     - Ілюстрація
     - Текст: “У вас ще немає подій”
     - Кнопка “Створити першу подію” (веде на F-1)

Навігація:

- Клік на картку події відкриває детальну аналітику (F-3)

Детально описано в розділі 5.4

F-3: Перегляд детальної аналітики події

Опис: При виборі конкретної події зі списку (F-2), організатор має доступ до розширеної панелі управління з аналітикою та функціями виведення коштів.

Передумови:

- Користувач підключив гаманець (F-0.1)

- Обрав роль “Організатор” (F-0.3)

- Є власником обраної події

Структура сторінки:

1) Блок статичної інформації, статистики події та посилань(детально – F-23).

2) Блок статистики продажу:

Система зчитує дані з смарт-контракту та індексера:

- **Первинний ринок:**
  - Дохід від первинного продажу: `12.5 ETH`
- **Вторинний ринок:**
  - Кількість перепродажів: `45 транзакцій`
  - Загальна сума роялті: `1.8 ETH`
- **Черги:**
  - Потенційний попит: “Якщо всі продажі виконаються, отримаєте додатково X роялті”
- **Загальний дохід:**
  - **Сумарний дохід:** `14.3 ETH` (первинний продаж + роялті)
  - Розбивка:
    - Первинний продаж: `12.5 ETH`
    - Роялті з вторинного ринку: `1.8 ETH`

3) Блок фінансів (залежить від фази події):

Якщо подія ще не завершилася (фаза 1 або 2):

- Відображається повідомлення: “Виведення коштів буде доступне після завершення події”

- Кнопки виведення **неактивні** (disabled) та показані сірим кольором

- Показується таймер: “До завершення: 5 днів 12 годин”

Якщо подія завершилася (фаза 3):

- Відображаються три кнопки виведення коштів (детально описані в F-4):

1. “Вивести дохід від первинного продажу”.

2. “Вивести роялті з вторинного ринку”.

3. “Вивести весь дохід” – доступна, якщо є хоча б один з балансів > 0

- Для кожної кнопки показується доступна для виведення сума
- Якщо певна категорія коштів вже виведена, відповідна кнопка стає неактивною з позначкою “✓ Виведено”

F-4: Виведення коштів організатором

Опис: Після завершення події організатор має можливість безпечно вивести зароблені кошти з балансу смарт-контракту на свій гаманець. Реалізовано патерн **“Pull over Push”** для мінімізації ризиків атак.

Передумови:

- Користувач підключив гаманець (F-0.1)

- Обрав роль “Організатор” (F-0.3)

– Є власником події

- Подія **завершилася** (поточний час > `endTime`)

- На балансі контракту є кошти для виведення

Доступні функції:

F-4.1: Виведення коштів від первинного продажу

- Користувач натискає кнопку “Вивести дохід від первинного продажу”
- Відображається підтверджуюче вікно:
  - Сума: `12.5 ETH`
  - Попередження: “Кошти будуть відправлені на адресу вашого гаманця”
- Після підтвердження формується транзакція, що викликає метод смарт-контракту для виводу доходу від первинного ринку.
- Транзакція підписується в гаманці
- Після підтвердження кошти надходять на гаманець організатора
- Інтерфейс оновлюється: кнопка стає неактивною з позначкою “✓ Виведено 12.5 ETH”

F-4.2: Виведення роялті з вторинного ринку

- Аналогічний процес для коштів, зароблених як роялті

F-4.3: Виведення всього доходу (комбінована функція)

- Зручна опція для виведення обох категорій коштів однією транзакцією
- Економить газ порівняно з двома окремими викликами
- Викликається метод для виводу усього доходу

Патерн CEI (Checks-Effects-Interactions): Смарт-контракт реалізує безпечний патерн:

1. **Checks:** Перевірка, що викликає – власник і подія завершилась

2. **Effects:** Обнулення балансу *до* відправки коштів (захист від reentrancy)

3. **Interactions:** Відправка коштів через `call`

Обробка помилок:

- "Подія ще не завершилася" – якщо час < `endTime`

- "Немає коштів для виведення" – якщо баланс = 0

- "Транзакція відхилена" – якщо користувач скасував підпис

- "Недостатньо газу" – якщо на балансі гаманця немає ETH для оплати комісії

Історія виведень:

- Після виконання виведення інформація записується в історію:

- Дата і час

- Сума

- Хеш транзакції

- Користувач може переглянути всі виведення на сторінці аналітики

5.2 Загальний функціонал покупця

Функції, доступні для ролі «покупець квитків».

F-5: Перегляд доступних подій

Опис: Покупець має можливість переглядати загальний каталог усіх подій, доступних у системі, незалежно від того, хто їх створив.

Передумови: - Користувач підключив гаманець (F-0.1) - Обрав роль "Покупець" (F-0.3)

Процес виконання:

1. **Запит даних:**

- Система виконує GraphQL-запит до індексера, отримуючи список усіх подій.

2. **Відображення:**
   - Події виводяться у вигляді **карток**
3. **Порожній стан:**
   - Якщо в системі взагалі немає подій:
     - Ілюстрація
     - Текст: “Поки що немає активних подій. Слідкуйте за оновленнями!”

Навігація:

- Клік на картку події відкриває детальну сторінку (F-23)

- З детальної сторінки доступні дії купівлі (F-8)

Детально описано в розділі 5.4

F-6: Купівля квитків на первинному ринку

Опис: Користувач може придбати один або кілька NFT-квитків безпосередньо від організатора за номінальною ціною, поки є вільні місця та подія не почалася. Це ключовий сценарій системи, який включає персоналізацію квитка через криптографічний підпис.

Передумови:

- Користувач підключив гаманець (F-0.1)

- На балансі гаманця достатньо коштів: `ціна_квитка × кількість + газ`

- Подія знаходиться у **фазі 1** (продаж квитків)

- Є вільні квитки: `soldTickets < maxSupply`

- Користувач не перевищив ліміт квитків на гаманець

- **Обмеження транзакції:** Кількість квитків за одну покупку не може перевищувати MAX_BATCH_SIZE (рекомендовано **5-10 одиниць**), щоб гарантувати успішне виконання транзакції в межах лімітів газу мережі.

Процес купівлі (покрокова інструкція):

Крок 1: Конфігурація замовлення

Користувач на детальній сторінці події (F-23):

1. **Обирає кількість квитків:**

- Через input-поле з кнопками +/-
- Система показує:
  - Доступно квитків: `250 / 500`
  - Ваш поточний ліміт: `4` (якщо вже є квитки, показується: “Куплено 2, доступно ще 2”)
- Якщо користувач намагається вибрати більше ліміту, показується попередження: “Перевищено максимальну кількість квитків на гаманець (4)”
- Максимальне значення в полі вводу не може бути більшим за менше з двох чисел: ліміт на гаманець або ліміт на одну транзакцію.
- Підказка в UI: «Ви можете придбати не більше 5 квитків за одну операцію для стабільної обробки транзакції блокчейном».

2. **Для кожного квитка вводить персональні дані:**

   Якщо купується **один квиток:**

   - Форма з двома полями(обов’язкові):
     - Ім’я латиницею (для уникнення проблем з розкладкою клавіатури на контрольних пунктах)
     - Прізвище латиницею (для уникнення проблем з розкладкою клавіатури на контрольних пунктах)

   Якщо купується **декілька квитків:**

   - Динамічна форма з секціями “Квиток 1”, “Квиток 2” тощо
   - Для кожного квитка окремі поля Ім’я та Прізвище

   Правила заповнення та підказки (UI Hints):

1. **Джерело транслітерації:**
   - *"Вводьте дані точно так, як у вашому закордонному паспорті або ID-картці."*
   - *Підказка:* "Якщо у вас немає закордонного паспорта, скористайтеся вашою **банківською карткою** – там ім'я завжди вказано латиницею. Це найкращий орієнтир."
2. **Відповідальність:**
   - *Попередження:* "Якщо ви виконуєте транслітерацію самостійно, ви зобов'язані запам'ятати (або зберегти), як саме ви ввели дані. На вході валідація пройде тільки при повному співпадінні."
3. **Люди з одним ім'ям:**
   - *Інструкція:* "Якщо у вас лише одне ім'я (без прізвища), введіть це ім'я **в обидва поля**: і в 'Ім'я', і в 'Прізвище'."

   Критично важливе попередження (показується червоним рамкою):

```
 УВАГА!
Вхід на захід буде можливий ТІЛЬКИ при наявності:
```

```
1. Документа, що посвідчує особу, з ТОЧНО такими ж ім'ям та прізвищем
2. Доступу до цього гаманця для генерації підтвердження володіння
гаманцем на вході на захід

Після покупки змінити або відновити ці дані НЕМОЖЛИВО, оскільки вони
зберігаються в зашифрованому вигляді (хеш) в блокчейні.
```

3. **Перегляд підсумків замовлення:**
   - Кількість квитків: `2`
   - Ціна за квиток: `0.05 ETH`
   - Сумарна вартість: `0.10 ETH`
   - Оцінка газу: `~0.003 ETH`
   - **Загальна вартість: `0.103 ETH`**
4. Натискає кнопку **“Продовжити до персоналізації”**

Крок 2: Персоналізація (криптографічний підпис)

Це найважливіший етап, що реалізує унікальну ідентифікацію власника.

1. **Формування повідомлення для підпису:**

   Для **кожного** квитка система автоматично генерує структуроване повідомлення згідно з **EIP-712** (Typed Data Signing):

```
// 1. Domain (Контекст безпеки)

const domain = {
  name: 'TicketPulse',
  version: '1',
  chainId: 1,
  verifyingContract: eventContractAddress
};

// 2. Types (Схема даних)

const types = {

// Назва структури, яку бачить користувач
  IdentityBinding: [
    { name: 'eventName', type: 'string' },
    { name: 'ownerFirstName', type: 'string' },
    { name: 'ownerLastName', type: 'string' }
  ]
};

// 3. Value (Конкретні дані для підпису)

const value = {
  eventName: 'The Neighbourhood - The World Tour 2026',
```

```
  ownerFirstName: 'Іван',
  ownerLastName: 'Петренко'
};
```

Такий підхід забезпечує:

- Читабельність: користувач **бачить**, що саме він підписує (гаманець відображає структуровані дані, а не просто хеш)
- Безпеку: підпис прив'язаний до конкретного контракту та мережі
- Стандартизацію: підтримується всіма сучасними гаманцями

2. **Запит підпису через гаманець:**

   Для **кожного** квитка:

   - Система викликає метод підпису
   - Гаманець відкриває модальне вікно з читабельною інформацією:

     ```
     Ви підписуєте право власності на квиток:

     Подія: The Neighbourhood - The World Tour 2026
     Власник: Іван Петренко

     Цей підпис не витрачає кошти. Він лише підтверджує вашу особу.
     ```

   - Користувач натискає "Підписати"
   - Отримується цифровий підпис у форматі:

     ```
     r: 0x1234...
     s: 0x5678...
     v: 27
     ```

     Це **65 байт** даних, що є криптографічним доказом

3. **Хешування підпису:**

   На стороні **клієнта** (а не в блокчейні) система обчислює **Keccak-256** хеш від бінарного представлення підпису:

   Цей хеш і буде записаний у смарт-контракт у логах. Це забезпечує:

   - **Конфіденційність:** персональні дані (ім'я, прізвище) ніколи не потрапляють у блокчейн
   - **Верифікацію:** маючи доступ до того ж гаманця та знаючи ті самі ім'я і прізвище, можна відтворити ідентичний підпис і хеш

4. **Підтвердження персоналізації:**

   - Користувач ставить галочку: "Я розумію, що ці дані незмінні"
   - Натискає **"Продовжити до оплати"**

Крок 3: Фінальна транзакція оплати

1. **Формування транзакції:**

   Система викликає метод смарт-контракту для мінтингу квитків:

```
function mintTickets(
    uint256 quantity,
    bytes32[] memory signatureHashes
) external payable {
    require(block.timestamp < startTime, "Sales ended");
    require(soldTickets + quantity <= maxSupply, "Not enough tickets");
    require(balanceOf(msg.sender) + quantity <= maxTicketsPerWallet,
"Limit exceeded");
    require(msg.value == nominalPrice * quantity, "Incorrect payment");

    for (uint256 i = 0; i < quantity; i++) {
        uint256 tokenId = _nextTokenId();
        _mint(msg.sender, tokenId);
        ticketPersonalization[tokenId] = signatureHashes[i];
        soldTickets++;
    }
}
```

   Параметри:

   - `quantity`: 2
   - `signatureHashes`: `[0xabcd..., 0xef12...]` – масив хешів для кожного квитка
   - `msg.value`: `0.10 ETH` (ціна × кількість)
2. **Підпис транзакції:**
   - Гаманець відкриває вікно підтвердження оплати
   - Показується сума та газ
   - Користувач підтверджує
3. **Очікування підтвердження:**
   - Транзакція в мемпулі
   - Інтерфейс показує: “Створення ваших квитків… ”
   - Посилання на block explorer
4. **Результат:**
   - Після підтвердження NFT-квитки з’являються на гаманці користувача
   - Показується повідомлення успіху:

```
 Вітаємо! Ви придбали 2 квитки на The Neighbourhood!

Квитки додано до вашої колекції.
```

- Система пропонує перейти до колекції квитків задля виконання необхідних дій з ними (F-7)

5. **Збереження підпису:**

   КРИТИЧНО ВАЖЛИВО:

   - Оригінальний **65-байтовий підпис** (набір байтів `r, s, v`) зберігається **локально** на пристрої користувача у LocalStorage браузера
   - Цей підпис стає **“Цифровим Артефактом”** – як файл ключа, який треба берегти
   - Ім’я та прізвища квитків, введених при купівлі квитків, також зберігається на пристрої користувача у LocalStorage браузера

Обробка помилок:

- “Не вистачає коштів для оплати” – якщо баланс < сума + газ
- “Перевищено ліміт квитків на гаманець” – якщо вже є максимальна кількість
- “Квитки розпродані” – якщо хтось інший купив останні квитки в той же момент
- “Невірний формат імені/прізвища” – якщо містять спецсимволи
- “Транзакція відхилена” – якщо користувач скасував підпис
- “Недостатня комісія газу” – якщо мережа перевантажена, система пропонує збільшити gas price або замістити транзакцію новою з тим самим nonce

---

F-7: Перегляд власної колекції квитків та управління ними

Опис: Покупець має доступ до особистого кабінету з повним списком усіх NFT-квитків, що належать його гаманцю. Квитки організовані у **5 секцій** за статусом, що автоматично змінюється залежно від дат події та дій користувача.

Передумови:

- Користувач підключив гаманець (F-0.1)

- Обрав роль “Покупець” (F-0.3)

Структура колекції (5 секцій):

Секція 1: Готові до використання

Опис: Квитки, якими користувач володіє і може використати для входу на захід. Це найчастіше використовувана секція.

Умови потрапляння квитка:

- Квиток належить користувачу (NFT на гаманці)

- Подія **ще не завершилася** (`currentTime <= endTime`).

- Квиток **НЕ виставлений на продаж**

- Якщо подія почалася, тоді квиток відображається у цьому списку, навіть якщо до цього був у «У черзі на продаж». Користувач отримує доступ до генерації QR-коду (F-9) без необхідності скасовувати заявку на продаж.

- Після завершення події квиток автоматично переходить у Секцію 5

Сортування:

- Автоматичне за датою початку івенту (ASC) – від найранішої до найпізнішої

- Це дозволяє користувачу бачити найближчі події спочатку

Відображення кожного квитка:

- **Мініатюра:** Обкладинка події

- **Назва події**

- **Тип квитка:** “VIP”, “General Admission”

- **Дата та час початку**

- **Місце проведення**

- **Статус:** “Готовий до використання” (зелений індикатор)

Дії (при кліку на квиток відкривається детальна картка):

Блок події:

- Клікабельна обкладинка (веде на F-23)

- Назва

- Опис події

- Дата і час початку та завершення

- Місце проведення

Блок активу:

- Тип квитка

- Ціна покупки: `0.05 ETH` (історична ціна, за яку куплено)

- Унікальний символ NFT-колекції: `TNWT26`

- Token ID: `#1234`

Блок управління:

1. **Кнопка “Генерувати QR-код для входу”** (веде на F-9):
   - Відкриває модальне вікно з **QR-кодом**
2. **Кнопка “Продати на вторинному ринку”** (ініціює F-14):
   - Доступна **тільки якщо подія ще не почалася** (`currentTime < startTime`)
   - Виставляє квиток у чергу на продаж за фіксованою ціною (номінал + націнка)
3. **Кнопка “Переглянути історію”** (веде на F-11):
   - Показує всіх попередніх власників квитка
4. **Кнопка “Додати в гаманець”** (веде на F-10):
   - Пропонує додати NFT у мобільний гаманець через `wallet_watchAsset` (EIP-747)

Пагінація:

- Load more (cursor pagination)

Пошук за назвою івенту

Секція 2: У черзі на продаж 💰

Опис: Квитки, які користувач виставив на продаж на вторинному ринку, але ще не були викуплені.

Умови потрапляння:

- Квиток належить користувачу

- Квиток **виставлений на продаж** (є в черзі FIFO у смарт-контракті)

- Подія **ще не почалася**

Сортування:

- Автоматичне за датою початку івенту (ASC)

Відображення:

- Обкладинка події

- Назва

- Дата

- Статус: “Чекає на покупця” (жовтий індикатор)

- **Позиція у черзі:** “Ви 3-й у черзі” (динамічно розраховується на основі часу створення заявки)

- **Очікувана ціна продажу:** `0.06 ETH` (номінал + націнка)

Дії (детальна картка):

- **Кнопка “Зняти з продажу”** (ініціює F-16):

- Видаляє квиток з черги

- Повертає статус “Готовий до використання”

- Квиток переміщується назад у Секцію 1

Пагінація та пошук: аналогічно Секції 1

Секція 3: Продані квитки (очікує виводу) 💸

Опис: Квитки, які були успішно продані через вторинний ринок, і кошти від продажу ще не виведені користувачем.

Умови потрапляння:

- Квиток **більше не належить користувачу** (передано новому власнику)

- На внутрішньому балансі смарт-контракту є кошти для виведення (продаж відбувся)

- Кошти **ще не виведені** (баланс > 0)

Сортування:

- Автоматичне за датою початку івенту (DESC)

Відображення:

- Обкладинка події

- Назва

- Дата

- Статус: “Продано” (зелений індикатор з галочкою)

- **Сума заробітку:** `0.054 ETH` (ціна продажу мінус роялті)

- Розбивка:

- Ціна продажу: `0.06 ETH`

- Роялті (10%): `-0.006 ETH`

- До виведення: `0.054 ETH`

Дії (детальна картка):

- **Кнопка “Вивести кошти”** (ініціює F-22):
  - Відкриває підтвердження
  - Формується транзакція виклику необхідного методу у смарт-контракті
  - Після підтвердження кошти надходять на гаманець
  - Квиток переміщується у Секцію 4

Пагінація та пошук: аналогічно попереднім секціям

Секція 4: Продані квитки (кошти виведено) ☑

Опис: Квитки, які були продані, і кошти від продажу вже успішно виведені.

Умови потрапляння:

- Квиток продано

- Кошти **виведено** (баланс = 0)

Сортування:

- Автоматичне за датою початку івенту (DESC)

Відображення:

- Обкладинка

- Назва

- Дата

- Статус: “Продано (кошти отримано)” (зелений індикатор)

- **Сума заробітку:** `0.054 ETH` (історична інформація)

- **Дата виведення:** `15.05.2026 14:32`

- **Хеш транзакції виведення** (посилання на block explorer)

Дії:

- Лише перегляд інформації

Примітка: Ця секція корисна для ведення власного обліку та звітності.

Пагінація та пошук: аналогічно попереднім секціям

Секція 5: Колекційні (подія завершена) 🏆

Опис: Квитки на події, які вже завершилися. Це цифрові колекційні артефакти, які зберігаються на гаманці назавжди.

Умови потрапляння:

- Квиток належить користувачу

- Подія **завершилася** (`currentTime > endTime`)

Сортування:

- Автоматичне за датою початку івенту (DESC) – найсвіжіші спогади спочатку

Відображення:

- Обкладинка (можливо з накладенням “Колекція”)

- Назва

- Дата проведення

- Статус: “Колекційний” (золотий/срібний індикатор)

Дії:

- **Перегляд деталей:**
  - Всі дані події (дата, місце, опис)
  - Історія володіння (F-11)
- **Програми лояльності (майбутня функція):**
  - Якщо організатор активував програму лояльності, користувач бачить:
    - “🎁 Доступні бонуси: Знижка 20% на наступний концерт артиста”
    - “🎁 VIP-доступ до зустрічі з артистом (для власників квитків)”
- **Підтвердження відвідування (майбутня функція):**
  - Якщо організатор синхронізував дані про використання квитків з офлайн-терміналів, квиток може мати позначку:
    - “✅Відвідано” (зелена позначка)
    - Або “Не використано” (сірий текст)
  - Це дозволяє організаторам винагороджувати саме тих, хто фізично був на заході

Пагінація та пошук: аналогічно попереднім секціям

Загальний функціонал для всіх секцій:

Пошук:

- Поле пошуку вгорі кожної секції

- Пошук за назвою івенту (входження підрядка)

Пагінація:

- Load more (cursor pagination)

- Підвантаження по 20 квитків за раз

- Індикатор завантаження

Порожній стан:

- Якщо в секції немає квитків:

- Ілюстрація

- Текст: “У цій секції поки що немає квитків”

Навігація між секціями:

- Табована навігація (tabs) вгорі: `[Готові (12)] [На продаж (3)] [Продані (5)] [Виведено (8)] [Колекція (15)]`

- Кількість у дужках – для зручності

Автоматичне переміщення квитків:

- Квитки автоматично переміщуються між секціями залежно від дій користувача та дат події

- Немає ручного керування секціями – все відбувається на основі стану смарт-контракту

F-8: Взаємодія зі сторінкою події

Опис: Користувач на детальній сторінці події (F-23) має доступ до набору інтерактивних елементів управління, які динамічно змінюються залежно від стану ринку та статусу користувача.

Передумови:

- Користувач підключив гаманець (F-0.1)

- Переглядає детальну сторінку події (F-23)

Логіка відображення кнопок:

Сценарій 1: Є вільні місця, подія не почалася

- **Умова:** `soldTickets < maxSupply` і `currentTime < startTime`

- **Кнопка:** “Купити квиток” (активна, яскрава)

- **Дія:** Відкриває форму купівлі (F-6)

Сценарій 2: Місця розпродані, подія не почалася

- **Умова:** `soldTickets >= maxSupply` і `currentTime < startTime` –

Кнопка: “Стати в чергу на купівлю” (активна), але ця кнопка також показується і при сценарії 1, 3, 4 та 5.

- **Дія:** Відкриває форму постановки в чергу (F-18)

- **Додаткова інформація:** “Вторинний ринок: 5 квитків у черзі на продаж”

Сценарій 3: Користувач у черзі на купівлю

- **Умова:** Адреса користувача є в черзі покупців для цієї події

- **Відображення:**

- Інформаційне повідомлення: “✓✓Ви 3-й у черзі на купівлю. Заблоковано 0.06 ETH ”

- Посилання: “Переглянути у своїй колекції заявок на купівлю” (веде на відповідну секцію F-13)

- Текст: “Для скасування заявки на купівлю квитка перейдіть до своєї колекції заявок”

Сценарій 4: Користувач є власником квитка на цю подію

- **Умова:** На гаманці користувача є NFT-квиток для цієї події

- **Відображення:**

- Інформаційне повідомлення: “✓✓У вас є квиток №X на цю подію. Виставлений на продаж на вторинному ринку (опціонально)”

- Посилання: “Переглянути у своїй колекції квитків” (веде на відповідну секцію F-7)

- Текст: “Для виставлення квитка на продаж або скасування продажу перейдіть до своїх квитків”

Сценарій 5: Квиток користувача було продано на вторинному ринку

- **Умова:** У смарт-контракті є кошти з продажу на вторинному ринку для виводу

- **Відображення:**

- Інформаційне повідомлення: “✓✓Ваш квиток №X було продано на вторинному ринку.”

- Посилання: “Переглянути у своїй колекції квитків” (веде на відповідну секцію F-7)

- Текст: “Для виведення коштів за квиток перейдіть до своїх квитків”

Сценарій 6: Подія вже почалася або завершилася

- **Умова:** `currentTime >= startTime` –

Відображення:

- Всі кнопки купівлі та постановки в чергу **неактивні** (disabled, сірі)

- Повідомлення:

- Якщо `currentTime >= startTime` і `currentTime < endTime`: “Подія зараз відбувається. Продаж квитків завершено.”

- Якщо `currentTime >= endTime`: “Подія завершилася. Квитки більше не продаються.”

F-9: Генерація динамічного QR-коду для входу на захід

Опис: Власник квитка генерує криптографічно захищений QR-код для проходження контролю. QR-код містить два шари даних: підтвердження особи (Identity) та підтвердження володіння гаманцем у реальному часі (Ownership).

Передумови:

- Користувач підключив гаманець (F-0.1)

- Володіє квитком (NFT на гаманці)

- Подія **ще не завершилася** (`currentTime <= endTime`)

- Локально збережено оригінальний підпис (Sig_A), згенерований при купівлі (F-6)

Процес генерації:

Крок 1: Ініціація

- Користувач відкриває детальну картку квитка (F-7)

- Натискає кнопку “Генерувати QR-код для входу”

- Система перевіряє наявність локального підпису (Sig_A). Якщо він відсутній, пропонується процедура відновлення (повторний підпис статичних даних). Але для цієї дії потрібно пам’ятати, яке ім’я та прізвище були введені при купівлі квитка.

Крок 2: Формування даних для QR-коду

QR-код містить **дві частини**:

Шар 1: Identity Proof (Статичний підпис – Sig_A)

- **Джерело:** LocalStorage (збережено в момент купівлі).
- **Суть:** Це цифровий підпис фіксованих даних: Event Name + Owner Name + Owner Surname.
- **Призначення:** Доводить, що людина з документами (наприклад, Іван Петренко) є саме тією особою, яка початково купила цей квиток. Цей підпис незмінний.

Шар 2: Ownership Proof (Динамічний підпис – Sig_B)

- **Джерело:** Генерується "тут і зараз" через гаманець користувача.
- **Механізм:** Користувач підписує поточний timestamp + ticketId.
- **Призначення:** Доводить, що користувач має активний доступ до приватного ключа гаманця в даний момент. Це унеможливлює використання чужого скріншоту, зробленого годину тому.

Процес створення Ownership Proof:

1. Система отримує поточний timestamp:

```
const currentTimestamp = Math.floor(Date.now() / 1000);
```

2. Формується повідомлення для підпису:

```
const message = ethers.utils.solidityKeccak256(
  ['uint256', 'uint256'],
  [ticketTokenId, currentTimestamp]
);
```

3. Запит підпису через гаманець:

```
const signature = await
signer.signMessage(ethers.utils.arrayify(message));
```

4. Підпис отримано у форматі 65 байт (r, s, v)

Крок 3: Процес підписання (Ручний режим)

У поточній версії відмовлено від автоматичної генерації кожні 30 секунд, оскільки більшість мобільних гаманців (MetaMask, Trust Wallet) вимагають явного підтвердження користувачем кожної операції підпису, що унеможливлює фоновий режим без додаткового функціоналу.

1. При відкритті модального вікна система показує запит: **"Підтвердіть генерацію коду доступу"**.
2. Користувач натискає "Підписати".
3. Гаманець відкриває вікно підпису повідомлення:

"Sign entry for Ticket #1234 at [Timestamp]"

4. Користувач підтверджує.
5. Отриманий підпис (Sig_B) додається до структури QR-коду.

Крок 4: Упакування даних у QR-код

Фінальний QR-код містить JSON-структуру:

```
{
  "version": 1,
  "ticketId": 1234,
  "eventId": 12345,
  "identityProof": {
    "signature": "0x1234abcd...", // Sig_A (65 bytes)
  },
  "ownershipProof": {
    "signature": "0x5678ef01...", // Sig_B (65 bytes)
    "timestamp": 1684843200 // Поточний час
  }
}
```

Ця структура кодується у QR-код (формат: QR Code Level M)

Крок 5: Інтерфейс відображення

Користувач бачить:

1. **QR-код:** Великий, контрастний, по центру.
2. **Таймер валідності:**
    - Зворотний відлік: *"Код дійсний: 01:59"*.
    - Час життя Sig_B встановлено на **5 хвилин**, що достатньо для проходження черги.
3. **Кнопка оновлення:**
    - Коли таймер спливає, QR-код розмивається (blur).
    - З'являється кнопка: "🔄 **Оновити код**".
    - Натискання ініціює повторний запит підпису (Крок 3).
4. **Інструкція для користувача:**

"Пред'явіть цей код та документ на ім'я: **Іван Петренко**".

Технічні обмеження та особливості

1. Робота без інтернету (Обмеження Web-версії)

У поточній реалізації веб-додатку (PWA/Browser):

- **Генерація:** Сам процес криптографічного підпису (signer.signMessage) виконується локально і **не потребує інтернету**.
- **Завантаження:** Однак, для відкриття сторінки сайту та завантаження скриптів JS потрібен доступ до мережі.
- **UI Рішення:** Якщо інтернет зникає *після* завантаження сторінки, генерація працюватиме. Якщо інтернету немає *до* входу на сайт — користувач не зможе відкрити інтерфейс.
- **Попередження:**

"⚠ Режим офлайн. Використовується кешована версія."

Майбутнє розширення: Повноцінний нативний мобільний додаток (iOS/Android) дозволить генерувати коди повністю офлайн без необхідності попереднього завантаження сторінки.

2. Захист від скріншотів (Anti-Replay)

- Валідатор перевіряє поле ts_gen (час генерації).
- Якщо (CurrentTime - ts_gen) > 5 хвилин — код вважається простроченим.
- Це означає, що скріншот, зроблений вдома перед виходом, не спрацює на вході. Код має бути "свіжим".

Валідація на вході (виконується контролером, опис у F-25):

1. Сканування QR-коду терміналом
2. Парсинг JSON-даних
3. Перевірка Identity Proof:
   - Обчислюється хеш від Sig_A
   - Порівнюється з хешем, записаним у блокчейні для ticketId
   - Якщо не збігається → вхід заборонено
4. Перевірка Ownership Proof:
   - Перевіряється, що timestamp свіжий (різниця < 5 хвилини)
   - Відновлюється адреса з підпису Sig_B
   - Порівнюється з поточним власником NFT (дані з snapshot бази)
   - Якщо не збігається → вхід заборонено
5. Криптографічна перевірка імені (Identity Check)

- Контролер бере фізичний документ (паспорт).
- Контролер вводить у термінал Ім'я та Прізвище з документа (наприклад, "Іван Петренко").
- Термінал автоматично формує повідомлення для підпису: Message = "EventName + Іван + Петренко".
- Термінал виконує математичне відновлення адреси з Sig_A (взятого з QR), використовуючи введені дані як повідомлення.
- Термінал перевіряє: Чи RecoveredSigner == CurrentTicketOwnerAddress?

Результат:

Якщо Так: Це математично доводить, що власник гаманця при покупці підписав саме ім'я "Іван Петренко". Вхід дозволено.

Якщо Ні: Це означає, що в Sig_A зашифровано інше ім'я (наприклад, "Петро"), і паспорт не відповідає квитку. Вхід заборонено.

6. Якщо всі перевірки пройдено → вхід дозволено, квиток позначається як використаний

Безпека:

- **Захист від скріншотів:** Динамічний Ownership Proof робить скріншот QR-коду непридатним через 5 хвилин
- **Захист від "продажу гаманця":** Навіть якщо зловмисник отримав доступ до приватного ключа, він не пройде валідацію документів на вході
- **Захист від копіювання NFT:** Навіть якщо хтось скопіює дані NFT, він не зможе підписати повідомлення без приватного ключа

F-10: Додавання квитка у мобільний гаманець

Опис: Користувач може додати NFT-квиток до свого Web3-гаманця для зручного відображення у застосунку гаманця. Це виконується через стандарт **EIP-747** (`wallet_watchAsset`).

Передумови:

- Користувач підключив гаманець (F-0.1)

- Володіє квитком

- Використовує, наприклад, мобільний гаманець (MetaMask Mobile, Trust Wallet, Rainbow тощо)

Процес:

1. Користувач натискає "Додати в гаманець" у детальній картці квитка (F-7)

2. Система формує запит `wallet_watchAsset`:

```
await window.ethereum.request({
  method: 'wallet_watchAsset',
  params: {
    type: 'ERC721',
    options: {
      address: eventContractAddress,
      tokenId: '1234'
    }
  }
});
```

3. Гаманець відкриває модальне вікно з запитом:

```
Додати NFT до гаманця?

Назва: The Neighbourhood - The World Tour 2026
Квиток #1234
Колекція: TNWT26
```

4. Користувач підтверджує

5. NFT з'являється в інтерфейсі гаманця з відображенням метаданих (обкладинка, назва, опис)

Важливе застереження:

Згідно з **EIP-747**, автоматичне додавання токена **без згоди користувача заборонено** протоколом безпеки. Це повинна бути явна дія користувача.

Обробка помилок:

- "Ваш гаманець не підтримує цю функцію" – якщо гаманець не реалізує EIP-747

- "Користувач відхилив запит" – якщо натиснуто "Скасувати"

Мета впровадження (Бізнес-логіка та UX)

Цей функціонал розроблений для вирішення кількох завдань взаємодії користувача з блокчейн-середовищем:

- **Гарантоване відображення активу (Подолання затримок індексації):** Оскільки для кожної події TicketPulse створює новий смарт-контракт, автоматичні сканери гаманців можуть не одразу розпізнати нову колекцію. Прямий виклик wallet_watchAsset змушує гаманець звернутися безпосередньо до контракту, миттєво показуючи квиток користувачу .

- **Коректна візуалізація та брендинг:** Без використання цього методу гаманець може відображати квиток як «Unknown Contract» (Невідомий контракт). Функція забезпечує правильне зчитування назви колекції (наприклад, TNWT26) та обкладинки через метадані, що підвищує довіру користувача до автентичності квитка .

- **Швидкий доступ (Native UX):** Користувачу зручніше перевірити наявність квитка у звичному інтерфейсі свого гаманця, ніж щоразу проходити авторизацію на сайті TicketPulse. Це особливо важливо в умовах черги на вході до заходу.
- **Підтвердження факту володіння:** Додавання квитка в особистий гаманець психологічно підтверджує успішність покупки та перехід права власності на цифровий актив (Digital Artifact) від організатора до покупця .
- **Безпека та контроль спаму:** Вимога явної дії користувача для додавання токена захищає інтерфейс гаманця від автоматичного захаращення небажаними активами (airdrop-спам), дотримуючись стандартів приватності Web3 .

F-11: Перегляд історії володіння квитком (chain of ownership)

Опис: Користувач може переглянути повну історію всіх транзакцій купівлі та перепродажу конкретного квитка з моменту його мінтингу. Це забезпечує прозорість та підтверджує автентичність.

Передумови: - Користувач переглядає детальну картку квитка (F-7)

Процес:

1. Користувач натискає “Переглянути історію” або “Chain of Ownership”
2. Система виконує запит до індексера (The Graph)
3. **Відображення хронології:**

   Формат: вертикальна Timeline

```
 Мінтинг квитка
├─ Організатор: 0x1234...abcd
├─ Ціна: 0.05 ETH (первинний продаж)
├─ Дата: 15.01.2026 10:30
└─ Транзакція: 0xabc123... [block explorer ↗]

↓

 Перепродаж #1
├─ Від: 0x5678...ef01
├─ До: 0x9abc...def2
├─ Ціна: 0.06 ETH (номінал + 20% націнка)
├─ Роялті організатору: 0.006 ETH (10%)
├─ Дата: 20.02.2026 14:15
└─ Транзакція: 0xdef456... [block explorer ↗]
```

```
↓

 Перепродаж #2
├─ Від: 0x9abc...def2
├─ До: 0x1234...5678 (Ви)
├─ Ціна: 0.06 ETH
├─ Роялті організатору: 0.006 ETH
├─ Дата: 01.03.2026 09:45
└─ Транзакція: 0x789abc... [block explorer ↗]
```

4. **Додаткова інформація:**
   - Загальна кількість передач: `3`
   - Загальна сума роялті, сплачена організатору: `0.012 ETH`

Використання:

- Доказ автентичності: перевірка, що квиток був замінтений офіційним контрактом - Прозорість: видно всіх попередніх власників

- Статистика: скільки разів квиток перепродавався

F-12:

Опис:

Передумови:

-

Процес:

1.

Використання:

F-13: Перегляд своїх заявок на купівлю

Опис: Користувач може переглядати всі свої заявки на купівлю квитків на вторинному ринку, відстежувати їх статус та управляти ними.

Передумови:

- Користувач підключив гаманець (F-0.1)

Структура:

Сторінка “/my-buy-requests” з двома секціями:

Секція 1: Активні заявки (невиконані) ⏳

Опис: Заявки, які ще чекають на виконання (не було автоматичного зіставлення з продавцем).

Сортування:

- Автоматичне за датою початку івенту (ASC) – найближчі події спочатку

Відображення:

- Обкладинка події

- Назва

- Дата та час початку

- **Ваша позиція у черзі:** “Ви 5-й у черзі на купівлю”

- **Заблоковані кошти:** `0.06 ETH`

- **Дата створення заявки:** `20.02.2026 14:30`

Дії:

1. **Кнопка “Скасувати заявку”** (ініціює F-20):
   - Видаляє заявку з черги
   - Повертає заблоковані кошти на гаманець

Пагінація:

- Load more (cursor pagination)

Пошук:

- За назвою івенту

Порожній стан:

- Якщо заявок немає:

- Ілюстрація

- Текст: “У вас немає активних заявок на купівлю”

- Кнопка: “Переглянути каталог подій”

Секція 2: Виконані заявки ☑

Опис: Заявки, які були успішно виконані (відбулося автоматичне зіставлення, квиток отримано).

Сортування:

- Автоматичне за датою початку івенту (ASC)

Відображення:

- Обкладинка

- Назва

- Дата

- **Ціна купівлі:** `0.06 ETH` (номінал + націнка)

- **Дата виконання:** `25.02.2026 10:15`

- **Хеш транзакції:** посилання на block explorer

Дії:

1. **Кнопка "Перейти до колекції квитків" (**F-7, Секція 1**)**

Пагінація та пошук: аналогічно

Порожній стан:

- Якщо заявок немає:

- Ілюстрація

- Текст: "У вас немає виконаних заявок на купівлю"

- Кнопка: "Переглянути список активних заявок"

5.3 Функціонал вторинного ринку

Функції, пов'язані з купівлею та продажем квитків на вторинному ринку після первинного продажу.

F-14: Виставлення квитка на продаж

Опис: Власник квитка може виставити **один** квиток за раз на вторинний ринок, додавши його до черги FIFO на продаж. Ціна продажу фіксована та розраховується автоматично смарт-контрактом.

Передумови:

- Користувач володіє квитком (NFT на гаманці)

- Квиток має статус “Готовий до використання” (F-7, Секція 1)

- Подія **ще не почалася** (`currentTime < startTime`)

Процес:

1. У детальній картці квитка (F-7) користувач натискає **“Продати на вторинному ринку”**
2. Відкривається підтверджуюче вікно з інформацією:

   ```
   Виставити квиток на продаж?

   Подія: The Neighbourhood - The World Tour 2026
   Квиток #1234

   Ціна продажу: 0.06 ETH
   (Номінал 0.05 ETH + 20% націнки)

   Розподіл коштів:
   - Роялті організатору (10%): 0.006 ETH
   - Ви отримаєте: 0.054 ETH

    Увага:
   - Ціна фіксована, торг неможливий
   - Якщо є покупець у черзі, продаж відбудеться миттєво
   - Квиток можна зняти з продажу до моменту автоматичного зіставлення
   ```

3. Користувач натискає **“Підтвердити”**
4. Формується транзакція виклику методу смарт-контракту
5. Користувач підписує транзакцію в гаманці
6. **Сценарій А: Миттєвий продаж (є покупці в черзі)**

   Якщо у черзі на купівлю є користувачі (F-18):

   - Система автоматично виконує зіставлення з **першим покупцем у черзі** (F-24)
   - NFT-квиток передається покупцю
   - Кошти розподіляються:
     - Роялті організатору записується на його внутрішній баланс
     - Решта коштів **автоматично відправляється прямо на гаманець продавця** в рамках цієї ж транзакції (метод Push допустимий, оскільки ініціатор = отримувач)

- Продавець отримує повідомлення:

```
 Ваш квиток продано!

Отримано: 0.054 ETH
Кошти вже на вашому гаманці.
```

- Квиток переміщується у F-7, Секція 4 (продано, кошти виведено)

7. **Сценарій B: Квиток додано в чергу (покупців немає)**

   Якщо черга покупців порожня:

   - Квиток додається до черги продавців (FIFO)
   - Квиток переміщується у F-7, Секція 2 (у черзі на продаж)
   - Відображається позиція у черзі: “Ви 3-й у черзі на продаж”
   - Продавець отримує повідомлення:

```
✅Квиток виставлено на продаж!

Ви 3-й у черзі на продаж.
Як тільки з'явиться покупець, продаж відбудеться автоматично.
```

Обробка помилок:

- “Подія вже почалася” – якщо час >= startTime

- “Транзакція відхилена”

F-15:

F-16: Зняття квитка з продажу

Опис: Власник може скасувати продаж квитка, поки він ще не був викуплений.

Передумови:

- Квиток у статусі “У черзі на продаж” (F-7, Секція 2)

- Автоматичне зіставлення ще не відбулося

- Подія не почалася

Процес:

1. У детальній картці квитка (F-7, Секція 2) користувач натискає **“Зняти з продажу”**
2. Відкривається підтвердження:

```
Зняти квиток з продажу?

Подія: The Neighbourhood
Квиток #1234

Квиток повернеться у статус "Готовий до використання".
```

3. Користувач підтверджує
4. Формується транзакція
5. Після підтвердження:
   - Квиток видаляється з черги продажу
   - Переміщується у F-7, Секція 1 (готовий до використання)

Обробка помилок:

- “Квиток вже продано” – якщо зіставлення відбулося між моментом натискання кнопки та підтвердженням транзакції

- “Подія почалася” – якщо час >= startTime

F-17:

F-18: Постановка в чергу на купівлю

Опис: Користувач, який бажає придбати квиток на подію, де первинні місця розпродані, може створити заявку на купівлю. Кошти блокуються на смарт-контракті до моменту виконання або скасування.

Передумови:

- Користувач підключив гаманець (F-0.1)

- Подія не почалася

- На балансі достатньо коштів: `(номінал + націнка) + газ`

- Користувач не перевищив ліміт квитків на гаманець

Процес:

1. На детальній сторінці події (F-23) користувач натискає **“Стати в чергу на купівлю”**
2. Відкривається форма (аналогічна F-6):

   Крок 1: Введення даних

   - Ім’я
   - Прізвище
   - Попередження про необхідність документа

   Крок 2: Персоналізація

   - Підпис повідомлення (EIP-712)
   - Хешування підпису
   - Збереження підпису локально

   Крок 3: Блокування коштів

   Підтверджуюче вікно:

   ```
   Створити заявку на купівлю?

   Подія: The Neighbourhood

   Ціна: 0.06 ETH
   (Номінал 0.05 + 20% націнка)

   Кошти буде заблоковано на смарт-контракті.
   Якщо з'явиться продавець, купівля відбудеться автоматично.

   Ви зможете скасувати заявку і повернути кошти у будь-який момент.
   ```

3. Користувач підтверджує
4. Формується транзакція
5. Підпис транзакції
6. **Сценарій А: Миттєва купівля (є продавці в черзі)**

   Якщо у черзі на продаж є квитки:

   - Система автоматично виконує зіставлення з **першим продавцем** (F-24)
   - NFT-квиток передається покупцю
   - Дані персоналізації оновлюються (хеш покупця замінює хеш продавця)
   - Кошти розподіляються:
     - Роялті → організатору

- Решта → продавцю

– Покупець отримує повідомлення:

```
 Ви придбали квиток!

Квиток додано до вашої колекції.
```

– Квиток з'являється у F-7, Секція 1

7. **Сценарій B: Додано в чергу (продавців немає)**

– Заявка додається до черги покупців (FIFO)

– Кошти заблоковано на контракті

– Заявка з'являється у F-13, Секція 1

– Повідомлення:

```
✅Заявка створена!

Ви 5-й у черзі на купівлю.
Як тільки з'явиться продавець, квиток автоматично перейде до вас.

0.06 ETH заблоковано на контракті.
```

F-19:

F-20: Вихід з черги на купівлю (скасування заявки)

Опис: Користувач може скасувати свою заявку на купівлю до моменту автоматичного зіставлення та повернути заблоковані кошти.

Передумови:

- Користувач має активну заявку (F-13, Секція 1)

- Зіставлення ще не відбулося

Процес:

1. У списку заявок (F-13, Секція 1) користувач натискає **"Скасувати заявку"**

2. Підтвердження:

```
Скасувати заявку на купівлю?

Подія: The Neighbourhood
```

```
Повернеться: 0.06 ETH
```

3. Користувач підтверджує
4. Формується транзакція
5. Після підтвердження:
   - Заявка видаляється з черги
   - Кошти повертаються на гаманець **у рамках цієї ж транзакції**

Важливо: Ця функція **залишається доступною завжди**, навіть під час і після події, для можливості повернення заблокованих коштів.

F-21:

F-22: Виведення коштів продавцем

Опис: Після успішного продажу квитка через чергу (зіставлення відбулося), продавець повинен ініціювати окрему транзакцію для отримання коштів з внутрішнього балансу контракту на свій гаманець.

Примітка: Якщо продаж відбувся миттєво (у момент виставлення, F-14), кошти вже на гаманці, і ця функція не потрібна.

Передумови:

- Квиток продано (F-7, Секція 3)

- На внутрішньому балансі контракту є кошти для виведення

Процес:

1. У детальній картці квитка (F-7, Секція 3) користувач натискає **“Вивести кошти”**
2. Підтвердження:

   ```
   Вивести кошти від продажу?

   Квиток: The Neighbourhood #1234

   Сума: 0.054 ETH
   (Ціна 0.06 - роялті 0.006)
   ```

3. Користувач підтверджує
4. Формується транзакція (патерн **Pull**).

5. Після підтвердження:
    - Кошти надходять на гаманець
    - Квиток переміщується у F-7, Секція 4

5.4 Вимоги до функціоналу списку подій

Стосовно F-2 та F-5, які описують відтворення списку подій для організатора та покупця відповідно, система повинна забезпечувати наступний функціонал:

1) Виведення списку подій

Реалізовано двома способами залежно від ролі:

Для покупця (F-5):

- **Механізм пагінації:** “Load more” (cursor pagination)

- При прокручуванні до кінця списку є можливість підвантажити наступні 20 подій

- Це забезпечує плавний UX без переходів між сторінками (безшовне підвантаження)

- Високу продуктивність при роботі з великими масивами даних

Для організатора (F-2):

- **Механізм пагінації:** Класична посторінкова навігація (offset pagination)

- Відображаються номери сторінок: `[1] [2] [3] ... [10]`

- Кількість подій на сторінку: 10

- Це дозволяє організатору швидко переходити до будь-якої сторінки

2) Відображення інформації про подію

Для кожної сутності у списку мають відображатися:

- **Обкладинка** (клікабельна)
- **Повна назва**
- **Місце проведення**
- **Дата і час початку** (у локальному часовому поясі користувача)
- **Номінальна ціна квитка** (формат “від X ”)
- **Візуальна позначка статусу:**
    - “Розпродано” – якщо `soldTickets >= maxSupply` і подія ще не почалася
    - “Доступні” – є вільні місця

3) Візуальна маркіровка подій за статусом часу

Окрім подій, які ще не розпочалися, система візуально маркує:

- **Події, що відбуваються в даний момент:**
  - Червона рамка або пульсуюча анімація
  - Текст: “ ЗАРАЗ”
- **Завершені події:**
  - Сіра напівпрозора накладка
  - Текст: “● Завершено”

4) Можливість пошуку за назвою події

- Поле пошуку вгорі списку
- Пошук за **входженням підрядка** (case-insensitive)
- Приклад: введення “neighbour” знайде “The Neighbourhood - The World Tour 2026”

5) Можливість фільтрації за діапазоном дати початку події

Поля фільтра:

- **“Дата початку події від”** *(обов’язкове)*:
  - Date picker
  - За замовчуванням: сьогоднішня дата
  - Валідація: не може бути пізніше за “до”
- **“Дата початку події до”** *(необов’язкове)*:
  - Якщо не вказано, показуються всі події після дати “від”

Поведінка за замовчуванням:

При першому завантаженні сторінки автоматично застосовується фільтр з **датою початку події від сьогодні**, тобто показуються **всі майбутні події**, включаючи сьогоднішні.

Приклад:

- Сьогодні: 09.12.2025

- Фільтр за замовчуванням: “від 09.12.2025”

- Показуються події з `startTime >= 09.12.2025 00:00:00`

6) Сортування за замовчуванням

Список автоматично сортується у **хронологічному порядку (ASC)**: від найранішої дати початку до найпізнішої.

Це дозволяє користувачам легко знаходити найближчі події спочатку списку.

5.5 Загальнодоступна інформація про подію

F-23: Детальна сторінка події та статистика

Опис: Єдина точка входу для отримання повної інформації про івент, доступна **як організаторам, так і покупцям** (публічна сторінка).

URL: `/event/[eventId]` або `/event/[contractAddress]`

Передумови:

- Подія створена (F-1)

- Сторінка доступна після входу як «Оргіназатор» або «Покупець»

Структура сторінки:

1) Блок статичної інформації:

- **Обкладинка** (великий формат, 1200x630 px)
- **Назва події**
- **Опис**
- **Місце проведення**
- **Дата і час початку** (з UTC конвертацією у локальний часовий пояс)
- **Дата і час завершення**
- **Статус події** (залежить від поточної дати):
  - – “Майбутня подія” (`currentTime < startTime`)
  - – “Подія відбувається” (`startTime <= currentTime < endTime`)
  - – ●“Подія завершена” (`currentTime >= endTime`)

2) Блок параметрів квитків:

- **Тип квитка:** “VIP”, “General Admission”
- **Символ NFT-колекції:** `TNWT26`
- **Номінальна ціна первинного продажу:** `0.05 ETH`
- **Правила вторинного ринку:**
  - – % роялті: `10%`
  - – % націнки: `20%`
  - – Ціна вторинного ринку: `0.06 ETH` (номінал + націнка)
- **Ліміт квитків на один гаманець:** `4`

3) Блок динамічної аналітики:

Первинний ринок:

- **Продано квитків:** `250 / 500` (з прогрес-баром)

- **Статус:**

- Якщо є вільні місця: “Доступно ще 250 квитків”

- Якщо розпродано: “Розпродано!”

Стан вторинного ринку:

- **Квитків у черзі на продаж:** 12

- **Покупців у черзі на купівлю:** 8

4) Блок функціональних дій (для навігації):

Для організатора:

- Можливість переглянути статистику продажів та вивести кошти (F-3 та F-4)

Для покупця:

- Відображення інтерактивних елементів згідно з F-8.

5) Блок додаткових посилань:

- **Посилання на смарт-контракт у block explorer:**
  - Приклад: “Переглянути контракт на Etherscan ↗”
- **Поділитися подією:**
  - Копіювати посилання

6) Таймер (якщо подія не завершилася):

- Якщо подія ще не почалася:
  - Великий таймер вгорі: “До початку: 5 днів 12 годин 34 хвилини”
- Якщо подія відбувається:
  - Анімація: “Подія зараз!” (пульсуючий текст)

7) Fallback Image (обробка помилок завантаження):

- Якщо обкладинка не завантажується з IPFS (тривале завантаження або помилка):
  - Відображається **картинка-заглушка** (placeholder)
  - Текст: “Завантаження обкладинки…” або “Обкладинка недоступна”
  - Реалізація патерну **“View Aggregator”**: спроба завантажити з декількох IPFS-гейтів (Pinata, Infura, Cloudflare)

5.6 Системна логіка (смарт-контракт)

F-24: Автоматичне зіставлення

Опис: Це внутрішня логіка смарт-контракту, яка спрацьовує автоматично при виконанні транзакцій F-14 (додавання квитка на продаж) або F-18 (додавання покупця в чергу). Якщо в **обох чергах** (на продаж і на купівлю) є учасники, система виконує атомарну операцію зіставлення.

Передумови (внутрішні перевірки контракту):

- `sellQueue.length > 0` і `buyQueue.length > 0`

- Подія не почалася (`currentTime < startTime`)

- Покупець не перевищив ліміт квитків на гаманець

Деталізація кроків:

1. Передача NFT-квитка:

- Квиток переміщується з гаманця продавця на гаманець покупця

- Змінюється `ownerOf(tokenId)`

2. Оновлення персоналізації квитка:

- **Старий хеш продавця** замінюється на **новий хеш покупця**

- Це критично важливо для валідації на вході (F-25)

- Покупець тепер може згенерувати валідний QR-код (F-9)

3. Розподіл заблокованих коштів покупця:

- **Роялті організатору:**
  - Відсоток від ціни вторинного продажу зараховується на **внутрішній баланс організатора**
  - Приклад: ціна `0.06 ETH`, роялті `10%` → `0.006 ETH`
  - Організатор зможе вивести роялті **виключно після завершення події** (F-4)
- **Решта суми продавцю:**
  - Зараховується на **внутрішній баланс продавця** для подальшого виведення (F-22)
  - Приклад: `0.06 - 0.006 = 0.054 ETH`

  Виняток (оптимізація UX): Якщо у черзі на купівлю є інші покупці (тобто `buyQueue.length > 1` після видалення поточного), кошти можуть бути **автоматично відправлені прямо на гаманець продавця** в рамках цієї ж

транзакції (метод **Push** допустимий, оскільки ініціатор транзакції є продавцем, що мінімізує ризики reentrancy)

4. Видалення обох учасників з відповідних черг:

- Продавець (ticketId) видаляється з `sellQueue`

- Покупець (адреса) видаляється з `buyQueue`

5. Емітування події для індексації

Це дозволяє The Graph індексувати транзакцію для відображення у статистиці (F-3, F-23, F-11).

Результат:

- **Продавець:**
  - Більше не володіє квитком
  - Має кошти на внутрішньому балансі (або вже отримав на гаманець)
  - Квиток переміщується у F-7, Секція 3 або 4
- **Покупець:**
  - Отримав NFT-квиток на гаманець
  - Квиток з'являється у F-7, Секція 1 (готовий до використання)
  - Може згенерувати QR-код (F-9)
- **Організатор:**
  - Отримав роялті на внутрішній баланс
  - Роялті доступні для виведення після завершення події (F-4)

Безпека (патерн CEI – Checks-Effects-Interactions):

Смарт-контракт дотримується безпечної послідовності:

1. **Checks:** Перевірка всіх умов (права власності, ліміти, час)

2. **Effects:** Зміна стану контракту (оновлення балансів, передача NFT)

3. **Interactions:** Зовнішні виклики (відправка коштів)

Це захищає від атак **reentrancy**.

5.7 Функціонал валідації та контролю доступу

Даний функціонал (F-25 та F-26) спроектований як частина **перспективного плану розвитку** платформи TicketPulse. У межах поточної реалізації TicketPulse основну увагу зосереджено на критично важливому ончейн-циклі: створенні подій, безпечному мінтингу та логіці вторинного ринку. Функціонал для контролерів винесено за межі поточної ітерації (MVP), оскільки він вимагає розробки окремого клієнтського ПЗ для терміналів контролю.

F-25: Офлайн-валідація квитків на вході

Опис: Механізм перевірки автентичності квитків та права власності на фізичному вході на захід, що працює **в офлайн-режимі** за допомогою попередньо завантаженого snapshot бази даних.

Передумови:

- Подія знаходиться у **фазі 2** (тривання події) або **фазі 3** (завершена)

- Snapshot бази даних був створений до початку події (F-26)

- Контролер на вході має пристрій з встановленим додатком валідації та завантаженою базою

Обладнання контролера:

- **Пристрій:** Планшет або смартфон (iOS/Android)
- **Сканер:** Вбудована камера для сканування QR-кодів
- **Додаток:** Офлайн-застосунок валідації з локальною БД

Процес валідації:

Крок 1: Власник квитка пред'являє QR-код

- Користувач відкриває додаток TicketPulse на своєму пристрої
- Генерує динамічний QR-код (F-9)
- Тримає екран перед сканером контролера

Крок 2: Сканування QR-коду

- Контролер сканує QR-код камерою

- Додаток парсить JSON-дані:

```
{
  "ticketId": 1234,
  "eventId": 12345,
  "identityProof": {
    "signature": "0x1234abcd...",
  },
  "ownershipProof": {
    "signature": "0x5678ef01...",
    "timestamp": 1684843200
  }
}
```

Крок 3: Перевірка Identity Proof (статичний підпис)

Мета: Довести, що дані власника відповідають запису в блокчейні.

1. Додаток обчислює **хеш від Identity Signature**:

```
const identitySignatureBytes =
ethers.utils.arrayify(identityProof.signature);
const computedHash = ethers.utils.keccak256(identitySignatureBytes);
```

2. Порівнює з **хешем, збереженим у snapshot БД** для `ticketId 1234`:

```
const storedHash = snapshotDB.getTicketHash(ticketId);
if (computedHash !== storedHash) {
    return "ВІДХИЛЕНО: Підпис не відповідає записаному";
}
```

Крок 4: Перевірка Ownership Proof (динамічний підпис)

Мета: Довести, що людина **прямо зараз** має доступ до гаманця.

1. **Перевірка свіжості timestamp:**

```
const currentTime = Math.floor(Date.now() / 1000);
const timeDiff = currentTime - ownershipProof.timestamp;
if (timeDiff > 300) { // 5 хвилин
    return "ВІДХИЛЕНО: QR-код застарів. Оновіть код.";
}
```

2. **Відновлення адреси з підпису:**

   Додаток відновлює адресу, яка підписала повідомлення:

```
const message = ethers.utils.solidityKeccak256(
    ['uint256', 'uint256'],
    [ticketId, ownershipProof.timestamp]
);
const recoveredAddress = ethers.utils.verifyMessage(
    ethers.utils.arrayify(message),
    ownershipProof.signature
);
```

3. **Порівняння з поточним власником:**

   Додаток отримує з snapshot БД адресу поточного власника квитка:

```
const storedOwner = snapshotDB.getTicketOwner(ticketId);
if (recoveredAddress.toLowerCase() !== storedOwner.toLowerCase()) {
    return "ВІДХИЛЕНО: Ви не є власником квитка";
}
```

Крок 5: Перевірка документа, що посвідчує особу

Цей етап доводить, що людина перед контролером — це саме той «Іван Петренко», який купив квиток.

1. **Звірка документів:** Контролер запитує фізичний документ (паспорт або ID-картку).

2. **Введення даних:** Контролер вводить Ім’я та Прізвище з документа латиницею у додаток валідації.
3. **Відновлення підпису (EIP-712):** Додаток бере identityProof.signature (Sig_A) із QR-коду та намагається «розкрити» його, використовуючи введені дані як ключ.
    - *Суть:* Якщо в Sig_A зашифровано «Ivan Petrenko», а контролер ввів «Petro Ivanov», математична функція ecrecover видасть помилкову адресу.
4. **Результат:** Якщо адреса збігається — особу власника підтверджено математично, навіть без інтернету.

Крок 6: Фінальна перевірка – чи використаний квиток

Цей крок гарантує, що квиток використовується лише один раз. Тут ми задіємо наш **Локальний вузол (Local Node)**.

1. **Локальна перевірка:** Додаток миттєво перевіряє прапорець isUsed у внутрішній пам'яті пристрою.
2. **Запит синхронізації (Network Sync):** Планшет надсилає швидкий запит до Локального сервера через Ethernet або Wi-Fi.
3. **Атомарне оновлення (Atomic Update):**
    - **Сценарій А (Зв'язок є):** Сервер перевіряє глобальний статус. Якщо квиток вільний, сервер ставить isUsed: true і повертає «Дозволено». Це відбувається за мілісекунди.
    - **Сценарій Б (Зв'язок відсутній - Failover):** Додаток переходить у режим **Offline-first**. Він пускає людину на основі локальних даних і ставить статус localStatus: "pending".
4. **Виявлення конфліктів (Conflict Resolution):** Як тільки зв’язок із сервером відновлюється, планшет «зливає» дані. Якщо виявиться, що за цей час квиток пройшов двічі на різних входах, система видасть сповіщення службі безпеки для реагування «по гарячих слідах».

Крок 7: Результат валідації

Якщо всі перевірки пройдено:

Додаток відображає:

```
✅ВХІД ДОЗВОЛЕНО!

Квиток: #1234
Власник: Іван Петренко
Подія: The Neighbourhood
Тип: VIP
```

- **Квиток позначається як використаний** у локальній БД(дані передаються на інші пристрої)
- **Звуковий сигнал:** “Успіх” (зелений екран)
- **Контролер пропускає власника**

Якщо хоча б одна перевірка провалена:

Додаток відображає:

```
❌ВХІД ЗАБОРОНЕНО!

Причина: [детальний опис помилки]
```

- **Звуковий сигнал:** “Помилка” (червоний екран)
- **Контролер не пропускає** та пропонує звернутися до стійки допомоги

Обробка помилок:

- “QR-код застарів” – власник має оновити код
- “Підпис недійсний” – можлива спроба підробки, відмова у вході
- “Документ не відповідає” – відмова, можливе шахрайство
- “Квиток вже використаний” – можлива спроба повторного входу або “продажу гаманця”

F-26: Створення snapshot бази даних перед подією

Опис: Автоматизований процес створення статичного зрізу (snapshot) стану блокчейну, який фіксує перелік **усіх квитків події**, їх **поточних власників** та **хешів персоналізації** на момент початку події. Цей snapshot завантажується на пристрої контролерів для офлайн-валідації (F-25).

Передумови:

- Подія переходить у **фазу 2** (тривання події)

- Поточний час `>= startTime`

- Усі ринкові операції заморожено

Процес:

Крок 1: Заморожування ринкових операцій

- Як тільки `block.timestamp >= startTime`, смарт-контракт автоматично блокує:
  - Первинну купівлю (F-6)
  - Виставлення на продаж (F-14)
  - Постановку в чергу на купівлю (F-18)
  - Зняття з продажу (F-16)
- Квитки, що вже були у черзі на продаж, **залишаються функціональними** для входу (генерація QR-коду доступна)

Крок 2: Індексація даних

Індексер (The Graph або backend-сервіс) виконує запит до блокчейну:

```
query CreateSnapshot($eventId: String!) {
  event(id: $eventId) {
    id
    name
    startTime
    endTime
    tickets {
      tokenId
      owner
      signatureHash
      isUsed
    }
  }
}
```

Отримується список **усіх квитків** з:

- Token ID

- Поточна адреса власника

- Хеш персоналізації

Крок 3: Формування snapshot файлу

Дані упаковуються у компактний формат (JSON або бінарний):

```
{
  "eventId": "12345",
  "snapshotTime": 1684756800,
  "tickets": [
    {
      "tokenId": 1234,
      "owner": "0x1234...abcd",
      "signatureHash": "0xabcd1234...",
      "isUsed": false,
    },
    ...
```

```
  ]
}
```

Крок 4: Завантаження snapshot організатором

- Snapshot файл автоматично стає доступним для завантаження організатором у F-3 (аналітика події)
- Кнопка: **“Завантажити базу для офлайн-валідації”**
- Формат: `.json` або `.db` (SQLite для швидкого пошуку)

Крок 5: Ініціалізація локального вузла.

- Організатор розгортає локальний сервер (ноутбук) у закритій мережі заходу.
- На сервер завантажується повний **Snapshot** бази даних із блокчейну.

Крок 6: Розподіл даних на пристрої.

- Планшети контролерів підключаються до локального сервера та завантажують копію бази у внутрішню пам'ять (SQLite).
- **Структура локального словника (Data Object):**

```
const ticketDatabase = {
  "1234": {
    owner: "0x1234...abcd",
    hash: "0xabcd1234...",
    isUsed: false,

    localStatus: synced/pending
  },
  ...
};
```

Механізм роботи "Real-time Sync":

- При скануванні пристрій спочатку перевіряє isUsed локально.
- Якщо false, пристрій відправляє запит на локальний сервер для підтвердження.
- **У разі успіху:** Сервер розсилає всім іншим планшетам сигнал оновити цей ticketId на isUsed: true з використанням WebSocket

Обробка відмови мережі (Failover):

- Якщо Wi-Fi "падає", додаток переходить у режим **Offline-first**.
- Валідація продовжується на основі локальних даних (математична перевірка підписів все ще працює!).

- Квиток маркується як localStatus: "pending". Як тільки зв'язок відновлюється, планшет "виштовхує" ці записи на сервер для синхронізації з іншими входами.

Синхронізація після події (опціонально):

Після завершення події організатор може:

- Зібрати дані про використання квитків з пристроїв контролерів

- Завантажити у блокчейн, хто фізично відвідав захід

- Використовувати для програм лояльності (F-7, Секція 5)